PROJORA – Technical Stack and Software Development Process

Job Interview Presentation | Full-Stack Web Application Development

Based on the actual development work I carried out on PROJORA, here is a professional presentation of the technologies, programming languages, development tools, and working process involved.

The most important thing to communicate in an interview is that PROJORA was not just a website design project. It was the development of a functional, database-connected, full-stack web application with authentication, data management, user permissions, privacy controls, and deployment.

FlowSync — SaaS Project Management Dashboard UI by Farzeen on Dribbble

dribbble.com

PROJORA

A Personal Project Library and Portfolio Management Web Application

Full-stack application

Next.js

Supabase

Netlify deployment

1. Front-end technologies and tools

The front end is the part of PROJORA that users interact with directly. It includes the interface, dashboard, navigation, forms, project cards, profile pages, and other visual elements.

A. Front-end programming languages

Language	How I used it
HTML	Structure and organization of web page content and elements.
CSS	Styling, visual presentation, spacing, layout, and responsiveness.
JavaScript	Interactive functionality and client-side application behavior.
TypeScript / TSX	Type-aware application development and React component implementation in the Next.js project.

B. Front-end frameworks and tools

[React Logo Vector PNG | Modern JavaScript UI Library Emblem - freepng]

1. React.js

Used to build reusable user-interface components, such as dashboard

sections, forms, project displays, and interactive elements.

Making a custom 404 page in Next.js - DEV Community

2. Next.js

my main front-end framework. It provided the application structure, routing, page organization, and integration between the front end and back end. I worked with Next.js 15.

TypeScript Logo Icons for Web and App Development

3. TypeScript

Used for structured, type-aware development of application components and functionality.

Front-end responsibilities I implemented included:

- Registration and login interfaces.
- User dashboard.
- Project creation and editing forms.
- Project catalogue management.
- User profile editing.
- Public profile and project pages.
- Project visibility controls.
- Contact information sharing controls.
- Navigation and protected application pages.

Technical accuracy note: React and Next.js are confirmed parts of my development stack. TypeScript/TSX is the language layer to identify in the project presentation; CSS is the styling technology. I would not claim a particular CSS framework such as Tailwind CSS without checking the project's dependencies.

2. Back-end technologies and tools

The back end is responsible for processing information, managing users, storing data, enforcing security, and communicating with the database.

A. Back-end programming languages

Language	How I used it
SQL	Creating database tables, defining relationships, configuring permissions, writing database migrations, and implementing security policies.
JavaScript / TypeScript	Application-side server functionality and communication with Supabase services.

B. Back-end frameworks, platforms and tools

Supabase - Review, Pricing & Alternatives | Delv

1. Supabase

my primary backend platform. I used it to manage authentication, database access, user information, project information, and security.

Postgresql Logo PNG Transparent With Clear Background ID 473068 | TOPpng

2. PostgreSQL

The relational database used through Supabase to store and organize application information.

3. Supabase Authentication

Used to implement account registration, login, user sessions, and authenticated access.

Unit testing Postgres with pgTAP. Like most professional developers, I… | by Simon McClive | Engineering on the incline | Medium

4. Row Level Security (RLS)

Used to protect users' data by enforcing database-level access policies. This ensures users can access and modify only the information they are authorized to manage.

Back-end functionalities I implemented included:

- User account authentication.
- Database creation and configuration.
- User profile data management.
- Project data storage.
- Catalogue creation and project assignments.
- Public and private project access.
- User-specific data ownership.
- Contact information privacy.
- Database security policies.
- Public profile data exposure through a controlled database view.

One important technical achievement was implementing a `public_profiles` database view with controlled exposure of profile information. This allowed public profile pages to display permitted information without exposing contact details that users had chosen to keep private.

3. Development, version control and deployment tools

Apart from programming languages and frameworks, I used additional tools to develop, manage, test, and publish PROJORA.

Tool	Practical purpose

Node.js	Runtime environment for the development tools and Next.js application.
npm	Installing and managing project dependencies and running development/ build commands.
Git	Version control and tracking source-code changes.
GitHub	Hosting the private project repository and managing source code.
Netlify	Hosting and deploying the web application.
Supabase Dashboard and SQL Editor	Configuring the backend, running migrations, and verifying database security.
ESLint	Identifying code-quality issues and enforcing code standards.
Windows Command Prompt / Terminal	Running commands, installing dependencies, and executing builds locally.

I also worked with environment variables to configure the connection between Next.js and Supabase securely.

Examples included:

- `NEXT_PUBLIC_SUPABASE_URL`
- `NEXT_PUBLIC_SUPABASE_ANON_KEY`
- `NEXT_PUBLIC_SITE_URL`

I also configured deployment environment variables on Netlify and excluded `.env.local` from the Git repository.

4. The practical working process: How I built PROJORA

This is particularly important during an interview because it demonstrates my understanding of the complete software development lifecycle.

my development workflow

1. Phase 1 – Idea generation and requirements analysis

   I began by identifying the problem and defining the purpose of PROJORA.

   I established that users needed a dedicated platform to organize and showcase completed projects, ongoing projects, and future projects.

   I then identified the main application requirements:

   - User registration and authentication.
   - Personal dashboards.
   - Project management.
   - Catalogue management.
   - Public project discovery.
   - Privacy and profile management.

2. Phase 2 – Technology selection and architecture

   I selected Next.js and React for the application interface and Supabase for backend services.

   I chose PostgreSQL as the database and designed the relationship between users, projects, catalogues, and project assignments.

   This established the foundation of the full-stack application.

3. Phase 3 – Front-end development

   I developed the user-facing application, including:

- Registration and login pages.
- Dashboard.
- Project management interfaces.
- Catalogue management interfaces.
- Profile editing.
- Public-facing pages. I structured the interface into reusable components and connected interactive features to application functionality.

4. Phase 4 – Database development

   I created the database structure using PostgreSQL through Supabase.

   I implemented four main database tables:

   - Profiles.
   - Projects.
   - Catalogues.
   - Project_catalogues. I also established relationships between the tables and executed SQL migrations to create the database structure.

5. Phase 5 – Backend integration

   I connected the Next.js application to Supabase.

   I implemented authentication, user-specific data retrieval, project creation, project editing, catalogue assignment, profile updates, and public profile access.

   I also configured environment variables to establish the connection between the application and backend services.

6. Phase 6 – Security implementation

   Security was a major part of the development process.

   I implemented Row Level Security policies to ensure that:

   - Users could manage their own projects.
   - Users could manage their own catalogues.

- Users could not access or modify another user's private data.
- Project assignments were restricted to authorized owners.
- Private projects were excluded from public discovery.
- Private contact information remained hidden unless the owner enabled sharing. I also created a controlled public profile view.

7. Phase 7 – Testing and quality assurance

   I tested the application using different user accounts and different access conditions.

   I checked:

   - Registration and login.
   - Project creation and editing.
   - Catalogue creation and assignment.
   - Data persistence after refreshing.
   - Public and private project visibility.
   - Profile editing.
   - Contact information privacy.
   - Cross-account data access.
   - Authentication route protection.
   - Database security policies. I also ran production builds and addressed code and configuration errors.

8. Phase 8 – Version control and deployment

   I used Git to track changes and GitHub to host the private source-code repository.

   I configured Netlify to deploy the Next.js application, set the necessary environment variables, and configured the production build process.

   This moved the application from local development toward production hosting.

5. PROJORA's system architecture

This is a simplified representation of how the technologies communicate with each other.

USER INTERFACE

HTML · CSS · JavaScript · TypeScript

React · Next.js

Application requests and responses

BACKEND SERVICES

Supabase · Authentication · Data access

Security policies · SQL

DATABASE

PostgreSQL

Profiles · Projects · Catalogues · Assignments

DEPLOYMENT AND SOURCE CONTROL
GitHub · Git · Netlify

6. Major features delivered and practical achievements

In an interview, I should explain what the technologies enabled me to accomplish rather than simply reciting their names.

Feature	Technical achievement
User authentication	Registration, login, and protected routes.
Project management	Creating, editing, and organizing project information.
Catalogue management	Creating collections and assigning projects to multiple catalogues.
Public Discover	Displaying public projects while hiding private ones.
User profiles	Editing and publishing personal profile information.
Privacy controls	Allowing users to control the visibility of contact information.
Database security	Implementing RLS and ownership-based access policies.
Deployment	Configuring GitHub and Netlify for application hosting.

Some particularly valuable results from my testing:

- I verified that two separate user accounts could not access each other's private projects and catalogues.
- I verified that catalogue assignments persisted after refreshing the application.
- I verified that private projects did not appear in public Discover.
- I verified that disabled contact-sharing settings prevented optional contact information from appearing publicly.
- I verified that signed-out visitors were redirected away from protected dashboard and profile-editing routes.

These are excellent examples to discuss because they show practical implementation and testing, not just theoretical knowledge.

7. Interview presentation script

Here is a professional way to explain the project to an interviewer. I can use this as my speaking script.

PROJORA – Full-Stack Web Application Development

Introduction

One of the projects I worked on was PROJORA, a full-stack web application designed to help professionals organize, manage, and showcase their completed, ongoing, and future projects.

The objective was to create more than a conventional portfolio website. I wanted to develop a dedicated project management and presentation platform where users could manage individual projects, organize them into catalogues, customize their profiles, and control what information they shared publicly.

Technologies and Programming Languages

For the front end, I worked with HTML, CSS, JavaScript, TypeScript, React, and Next.js. These technologies enabled me to develop the user interface, interactive components, dashboard, forms, project pages, and profile management features.

For the back end, I used Supabase as the backend platform, PostgreSQL as the relational database, and SQL for database creation, relationships, migrations, and security policies. I also used JavaScript and TypeScript for application-side integration with backend services.

Other tools included Node.js, npm, Git, GitHub, Netlify, Supabase Dashboard, and ESLint.

Development Process

I approached the project through several development stages.

First, I identified the problem the application needed to solve and translated the idea into functional requirements.

Second, I selected the technology stack and designed the application architecture, including the database structure and relationships.

Third, I developed the front-end interface using React and Next.js.

Fourth, I designed and implemented the PostgreSQL database using Supabase and SQL.

Fifth, I integrated the front end with the backend, implementing authentication, project management, catalogue management, and profile functionality.

One of the most important parts of the project was security. I implemented Row Level Security policies to ensure that users could only manage their own private data. I also introduced privacy controls that allowed users to decide which profile and contact information could be displayed publicly.

After implementation, I carried out functional testing, database security testing, and cross-account testing using separate user accounts.

Finally, I used Git and GitHub for version control and configured Netlify for deployment.

Technical Challenges and Learning Experience

One of the important technical challenges was ensuring that the application did not simply display data correctly but also protected that data at the database level.

I addressed this by implementing ownership-based Row Level Security policies and testing access restrictions using separate user accounts.

Another important aspect was connecting the front end, authentication system, database, and deployment environment so that the application worked as an integrated system.

Conclusion

Working on PROJORA gave me practical experience with full-stack web application development, frontend and backend integration, relational database design, authentication, data privacy, security implementation, testing, version control, and deployment.

It also strengthened my understanding of how to transform an idea into a functional software product through structured development and continuous testing.

8. Quick interview revision sheet

I can use this table to revise the technologies before my interview.

Interview question	My answer
What did you build?	PROJORA, a full-stack project management and portfolio platform.
What did you use for the front end?	React and Next.js, with HTML, CSS, JavaScript and TypeScript.
What did you use for the back end?	Supabase and PostgreSQL.
What database language did you use?	SQL.
How did you authenticate users?	Supabase Authentication.
How did you protect user data?	PostgreSQL Row Level Security policies.
How did you manage source code?	Git and GitHub.
How did you deploy the application?	Netlify.

How did you test it?	Functional testing, database checks, cross-account security testing, and production builds.

9. My key interview points

There are three things I would emphasize when presenting PROJORA:

1. Full-stack development: I worked across the front end, back end, database, and deployment rather than concentrating on only one part of a website.

2. Database security: I implemented actual database-level security using PostgreSQL RLS, ownership policies, and controlled public profile data. This is a particularly valuable technical achievement.

3. End-to-end product development: I took an idea, translated it into requirements, selected technologies, built functionality, tested it with different accounts, and configured deployment.

One important distinction when describing my experience: Be confident about the implementation work you personally performed, but distinguish between independently writing every line of code and using AI coding assistance to help develop, debug, and implement the application. Both can demonstrate valuable software development skills when explained honestly.

The strongest way to describe my role is:

"I worked on the design, development, backend integration, database architecture, security implementation, testing, and deployment of a full-stack web application called PROJORA."

That communicates considerably more technical value than simply saying, "I created a website.